

Konnektortypen und Ingestion-Modi im Überblick

Section 2 · Data Ingestion and Loading · Quelle: Udemy-Kursmaterial „Lakeflow Connect“, ergänzt mit Databricks-Dokumentation

Die vorangegangenen Kapitel dieser Section haben einzelne Ingestion-Techniken im Detail behandelt – CTAS, COPY INTO, Auto Loader (Kapitel 1 & 6) sowie Managed Connectors und Partner Connect (Kapitel 4). Dieses Kapitel liefert die **übergeordnete Landkarte** dazu: die offizielle Einteilung in **vier Konnektortypen** und **drei Ingestion-Modi**, wie sie auch im Exam Guide verwendet wird – inklusive einer Entscheidungsregel, wann welcher Konnektortyp sinnvoll ist.

1. Vier Konnektortypen von Lakeflow Connect

Lakeflow Connect gliedert sämtliche Ingestion-Wege in vier Kategorien, jede für einen anderen Anwendungsfall optimiert:

Konnektortyp	Charakteristik	Details in
Managed Connectors	Out-of-the-box, low-code/no-code, für konkrete Enterprise-Quellen (Salesforce, Workday, SQL Server, ...)	Kapitel 4 dieser Section
Standard Connectors	Anpassbar, erfordern eigenen Python-/SQL-Code (Auto Loader, COPY INTO, CTAS, Structured Streaming)	Kapitel 1 & 6 dieser Section
Partner Connect	Drittanbieter-Dienste (z. B. Fivetran, Qlik, Informatica) für Quellen ohne eigenen Managed Connector	Kapitel 4 dieser Section
Manual File Upload	Lokale Dateien direkt über die Databricks-Oberfläche hochladen	siehe Abschnitt 3 unten (neu)

Connector types

- ▶ Lakeflow Connect provides multiple types of connectors, each optimized for specific source categories and ingestion patterns:
 - ▶ Managed connectors: Out-of-the-box connectors
 - ▶ Standard connectors: Customizable connectors
 - ▶ Partner Connect: 3rd-party ingestion services
 - ▶ Manual File upload

Derar Alhussein © Udemy | Databricks Certified Data Engineer Associate - Preparation

Lakeflow Connect bietet vier Konnektortypen, jeweils optimiert für unterschiedliche Quellkategorien und Ingestion-Muster.

2. Drei Ingestion-Modi im Überblick

Unabhängig vom gewählten Konnektortyp lässt sich jede Ingestion einem von drei Mustern zuordnen – eine Unterscheidung, die im Exam Guide explizit als „batch, streaming, and incremental loading“ benannt wird:

2.1 Batch

Daten werden als abgeschlossene Charge geladen, oft zeitgesteuert – bei jedem Lauf wird der **gesamte** Datensatz aus der Quelle erneut verarbeitet.

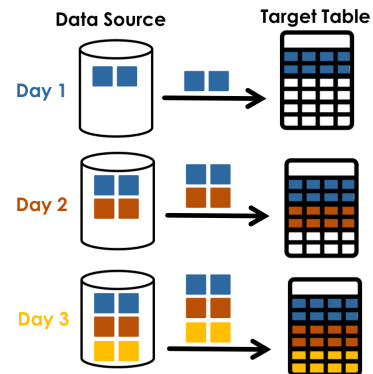
```
-- SQL: CREATE TABLE AS SELECT (CTAS)
CREATE TABLE bronze_orders AS SELECT * FROM read_files('/Volumes/.../orders', format =>
'csv');
```

```
# Python: einmaliges Batch-Read
df = spark.read.load("/Volumes/.../orders")
```

Batch

- ▶ Load data as batches of rows, often based on a schedule
- ▶ Reprocess all records with each run
- ▶ Techniques:
 - ▶ **SQL:** CREATE TABLE AS SELECT (CTAS)
 - ▶ **Python:** spark.read.load()

Derar Alhussein © Udemy | Databricks Certified Data Engineer Associate - Preparation



Batch-Ingestion: Bei jedem Lauf wird der komplette Datenbestand der Quelle erneut in die Zieltabelle geladen.

2.2 Incremental Batch

Nur **neu hinzugekommene** Datensätze werden geladen, bereits verarbeitete Dateien automatisch übersprungen – schneller und ressourcenschonender als klassisches Batch.

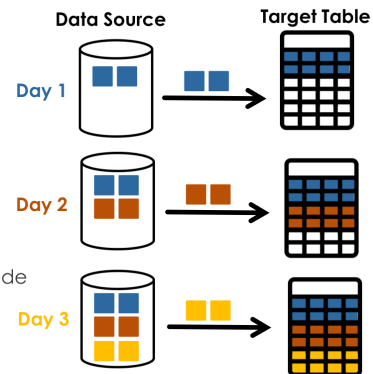
```
-- SQL: COPY INTO (idempotent)
COPY INTO bronze_orders FROM '/Volumes/.../orders' FILEFORMAT = CSV;
```

```
# Python: Spark Structured Streaming im Trigger-Modus (einmalig ausführen, dann stoppen)
(spark.readStream.format("cloudFiles").option("cloudFiles.format", "csv")
 .load("/Volumes/.../orders")
 .writeStream.trigger(availableNow=True)
 .toTable("bronze_orders"))
```

In Lakeflow Declarative Pipelines (SDP) entspricht dies einer Streaming Table im **getriggerten Pipeline-Modus** (die Pipeline läuft, verarbeitet neue Daten und stoppt wieder, statt dauerhaft zu laufen).

Incremental Batch

- ▶ Only new data is ingested
- ▶ Previously loaded records are skipped
- ▶ Faster and efficient ingestion
- ▶ Techniques:
 - ▶ **SQL:** COPY INTO
 - ▶ **Python:** Spark Structured Streaming in triggered mode
 - ▶ spark.readStream & Auto Loader
 - ▶ **SDP:** Streaming tables in triggered pipeline mode



Derar Alhussein © Udemy | Databricks Certified Data Engineer Associate - Preparation

Incremental Batch: Nur die seit dem letzten Lauf neu hinzugekommenen Datensätze werden geladen, bereits verarbeitete Daten bleiben unangetastet.

2.3 Streaming

Daten werden **kontinuierlich** in sehr kurzen, häufigen Intervallen (Sekunden bis Minuten) als Micro-Batches verarbeitet – ideal für Quellen wie Apache Kafka oder Amazon Kinesis, bei denen nahezu Echtzeit-Aktualität gefragt ist.

```
# Python: Spark Structured Streaming im kontinuierlichen Modus (läuft dauerhaft)
(spark.readStream.format("cloudFiles").option("cloudFiles.format", "json")
  .load("/Volumes/.../events")
  .writeStream.toTable("bronze_events"))
```

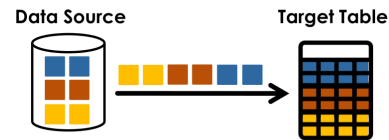
In SDP entspricht dies einer Streaming Table im **kontinuierlichen Pipeline-Modus**.

Streaming

- ▶ Continuously load micro batches of data at very short, frequent intervals
- ▶ Ideal for streaming sources such as
 - ▶ Kafka, Amazon Kinesis, Google Pub/Sub, ...

- ▶ Techniques:

- ▶ **Python:** Spark Structured Streaming in continuous mode
 - ▶ `spark.readStream` & Auto Loader
- ▶ **SDP:** Streaming tables in continuous pipeline mode



Derar Alhussein © Udeemy | Databricks Certified Data Engineer Associate - Preparation

Streaming-Ingestion: Daten werden fortlaufend in kleinen Micro-Batches verarbeitet, sodass sie nahezu in Echtzeit abfragbar sind.

3. Manual File Upload

Für einmalige, kleine Datenmengen – etwa eine lokale CSV-Datei zum Ausprobieren – lässt sich eine Datei auch ganz ohne Code direkt über die Databricks-Oberfläche hochladen. Dabei stehen zwei Ziele zur Wahl: die Datei kann direkt in eine neue **Delta-Tabelle** importiert werden, oder unverändert in ein **Unity-Catalog-Volume** abgelegt werden, um sie von dort aus z. B. per `read_files()` weiterzuverarbeiten.

4. Welchen Konnektortyp wählen? Die Prioritätsregel

Bei mehreren infrage kommenden Optionen empfiehlt Databricks, sich **von der am stärksten verwalteten Ebene abwärts** vorzuarbeiten und erst dann eine Ebene tiefer zu gehen, wenn die aktuelle Ebene die Anforderungen nicht erfüllt oder die Datenquelle nicht unterstützt:

1. **Managed Connector** prüfen – existiert einer für die konkrete Quelle (Salesforce, SQL Server, ...)? Wenn ja, verwenden.
2. **Standard Connector** (Auto Loader für Dateien, Structured Streaming für Kafka/Kinesis) – wenn kein Managed Connector existiert, aber die Quelle Cloud-Speicher oder ein Message-Bus ist.
3. **Partner Connect** – wenn weder Managed noch Standard Connector die Quelle abdecken, aber ein Technologiepartner (z. B. Fivetran) dies leistet.

`COPY INTO` bleibt dabei als einfacher, transaktionaler Batch-Befehl vor allem für Ad-hoc-Backfills und klar abgegrenzte historische Ladevorgänge sinnvoll, während `CREATE STREAMING TABLE` mit `read_files()` für SQL-Nutzer inzwischen als die skalierbarere, empfohlene Alternative für inkrementelle Ingestion gilt.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks fasst Lakeflow Connect als dreistufiges Modell zusammen: vollständig verwaltete Connectors (Managed), Standard-Connectors für Cloud-Speicher und Message-Busse (mit Auto Loader als zentralem Baustein für Datei-Ingestion), und Partner-Lösungen als dritte Ebene. Die Empfehlung lautet ausdrücklich, zunächst die am stärksten verwaltete Ebene zu prüfen und erst bei Nichteignung eine Ebene tiefer zu gehen – genau die Priorisierungslogik, die auch der Exam Guide unter „Prioritize between Auto Loader, Lakeflow Connect (...), and other ingestion methods“ abfragt.

Quelle: [What is Lakeflow Connect? – Databricks-Dokumentation](#) · [Load data in pipelines – Databricks-Dokumentation](#)